



FACULTY OF ELECTRONICS AND COMPUTER TECHNOLOGY AND ENGINEERING

SEM II - 2025/26 BERR 2243
DATABASE AND CLOUD SYSTEM

EXERCISE 3 REPORT

LECTURER NAME:
PROFESOR MADYA DR SOO YEW GUAN

No.	Student Name	Matrix No.
1	IKMAL IMAN BIN MOHD YUSOF	B122410306
2	MUHAMMAD ARIF SYAFIE BIN AGUS SALEH	B022210116
3	MUHAMMAD HAIKAL BIN ZULKIFLI	B122410348

Step 1

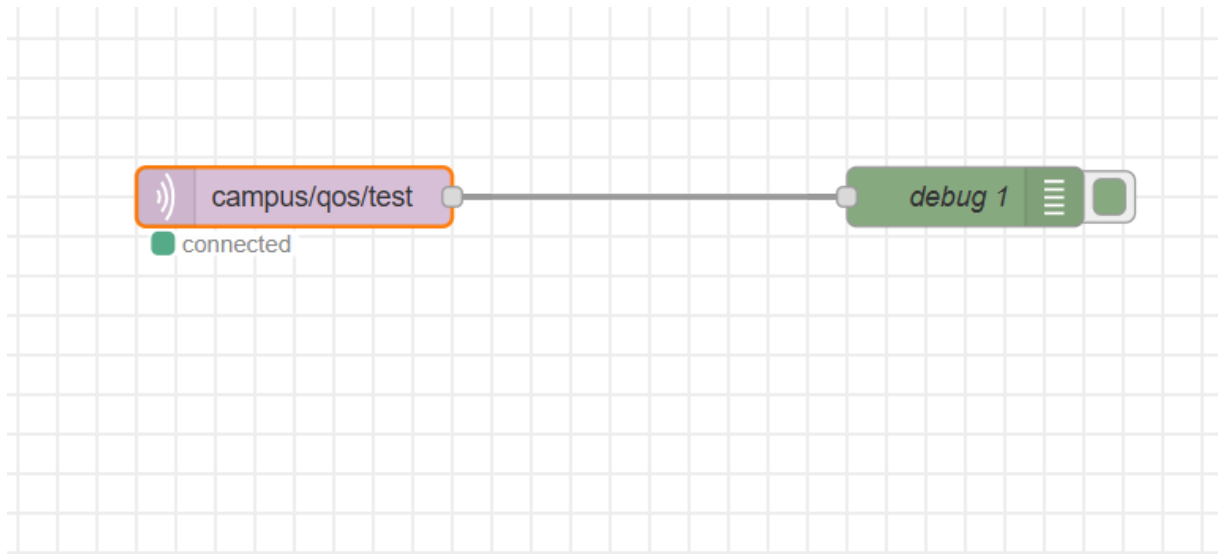


Figure 1: Nodered Connection

Step 2

QoS 0

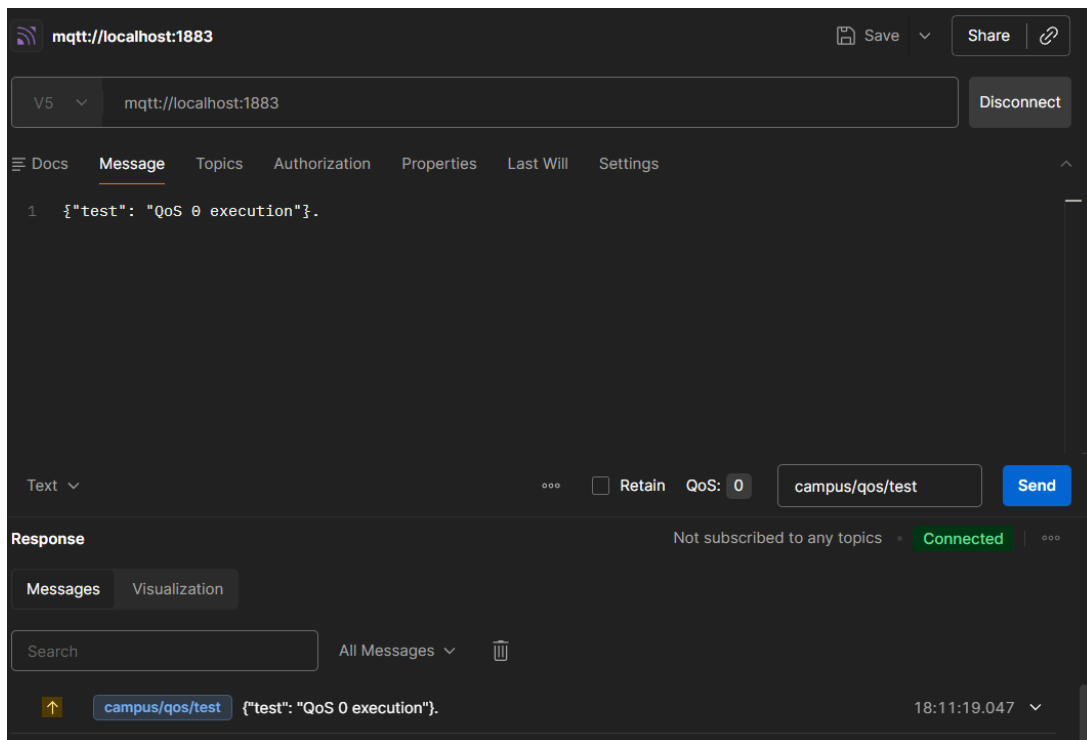


Figure 2 : Postman Configuraton

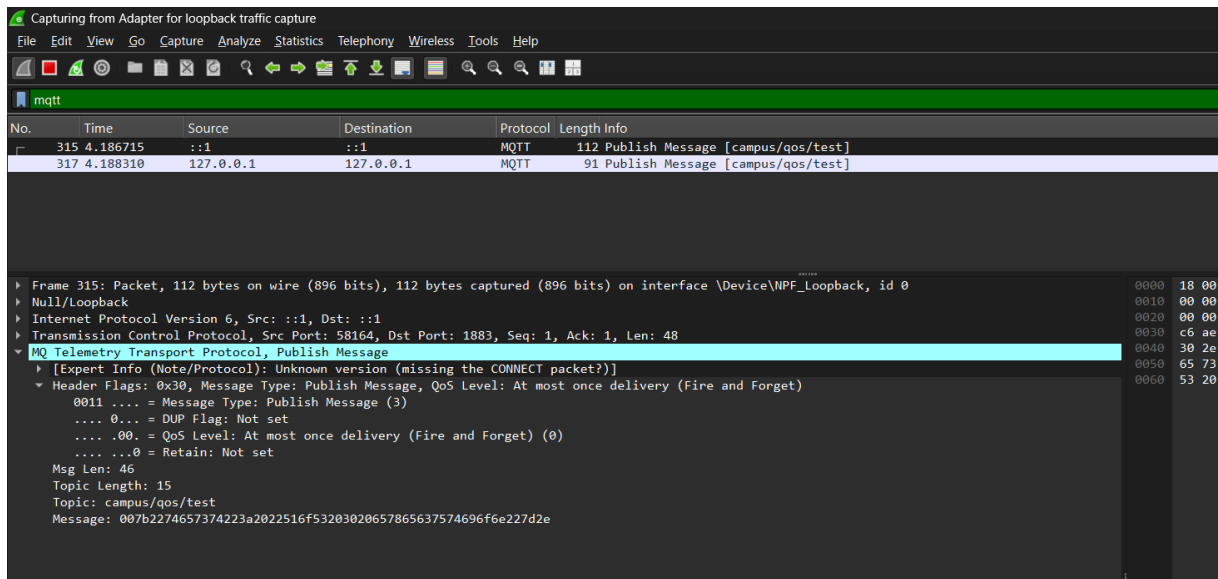


Figure 3: Wireshark Traffic

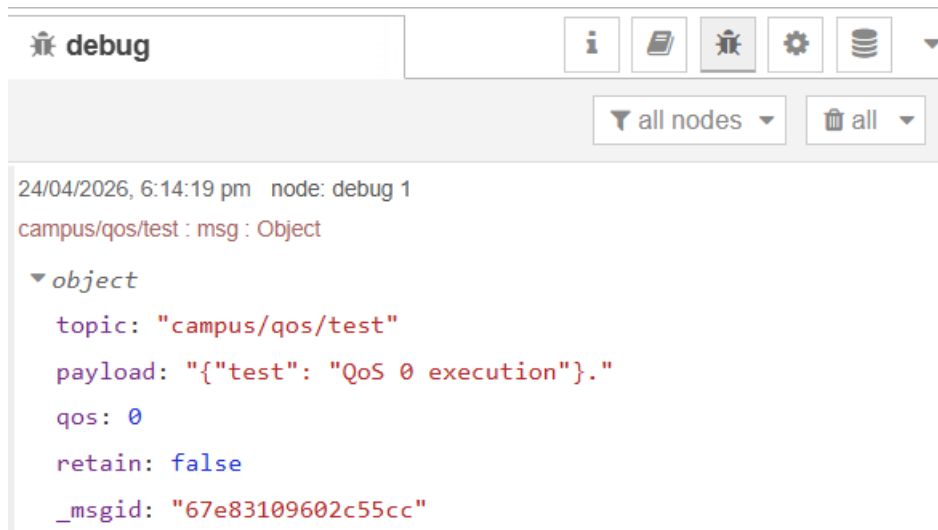


Figure 4 : Nodered Debug input

Step 3:

QoS 1 Execution

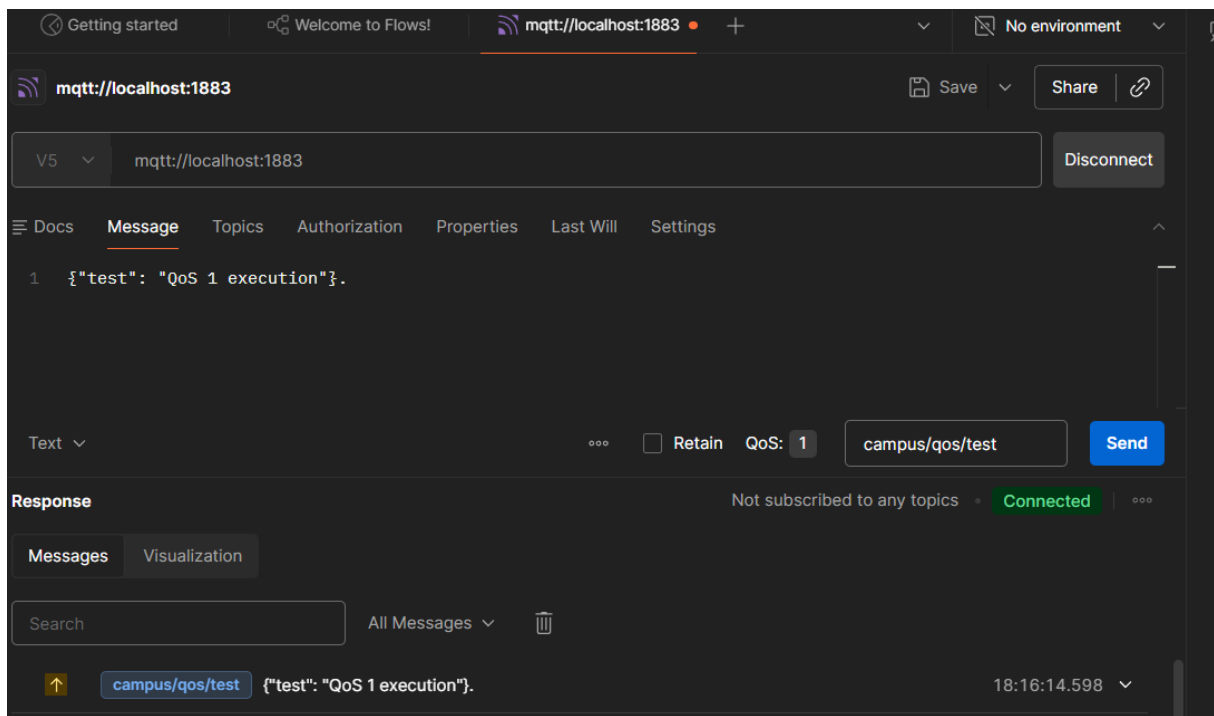


Figure 5 : Postman Configuraton

The screenshot shows a Wireshark traffic capture of MQTT messages. The interface includes a menu bar (File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, Help) and a toolbar. The main pane displays a list of captured packets:

No.	Time	Source	Destination	Protocol	Length	Info
417	31.670586	:::1	:::1	MQTT	114	Publish Message (id=4553) [campus/qos/test]
419	31.671702	:::1	:::1	MQTT	70	Publish Ack (id=4553)
421	31.671832	127.0.0.1	127.0.0.1	MQTT	93	Publish Message (id=2) [campus/qos/test]
423	31.673277	127.0.0.1	127.0.0.1	MQTT	48	Publish Ack (id=2)

Figure 6: Wireshark traffic

```

24/04/2026, 6:17:59 pm  node: debug 1
campus/qos/test : msg : Object
  object
    topic: "campus/qos/test"
    payload: '{"test": "QoS 1 execution"}.'
    qos: 1
    retain: false
    _msgid: "840bc67aacf2ab37"

```

Figure 7: Nodered Debug Output

QoS 2

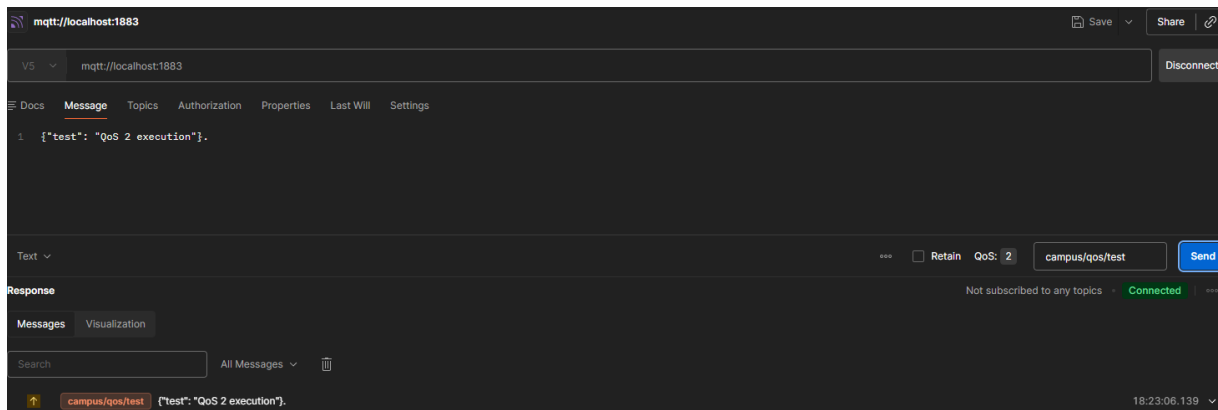


Figure 8: Postman Configuration

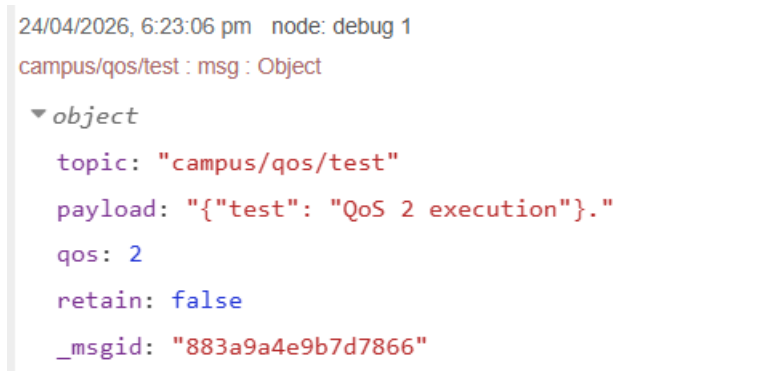


Figure 9: Nodered Debug

Capturing from Adapter for loopback traffic capture					
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help					
mqtt					
No.	Time	Source	Destination	Protocol	Length Info
91	3.767345	:::1	:::1	MQTT	114 Publish Message (id=3079) [campus/qos/test]
93	3.768026	:::1	:::1	MQTT	70 Publish Received (id=3079)
95	3.768519	127.0.0.1	127.0.0.1	MQTT	93 Publish Message (id=5) [campus/qos/test]
97	3.770211	127.0.0.1	127.0.0.1	MQTT	48 Publish Received (id=5)
99	3.771306	127.0.0.1	127.0.0.1	MQTT	48 Publish Release (id=5)
101	3.772860	127.0.0.1	127.0.0.1	MQTT	48 Publish Complete (id=5)
103	3.782605	:::1	:::1	MQTT	70 Publish Release (id=3079)
105	3.783032	:::1	:::1	MQTT	70 Publish Complete (id=3079)

Figure 10: Wireshark Traffic

Step 4: The Retain Flag Mechanism

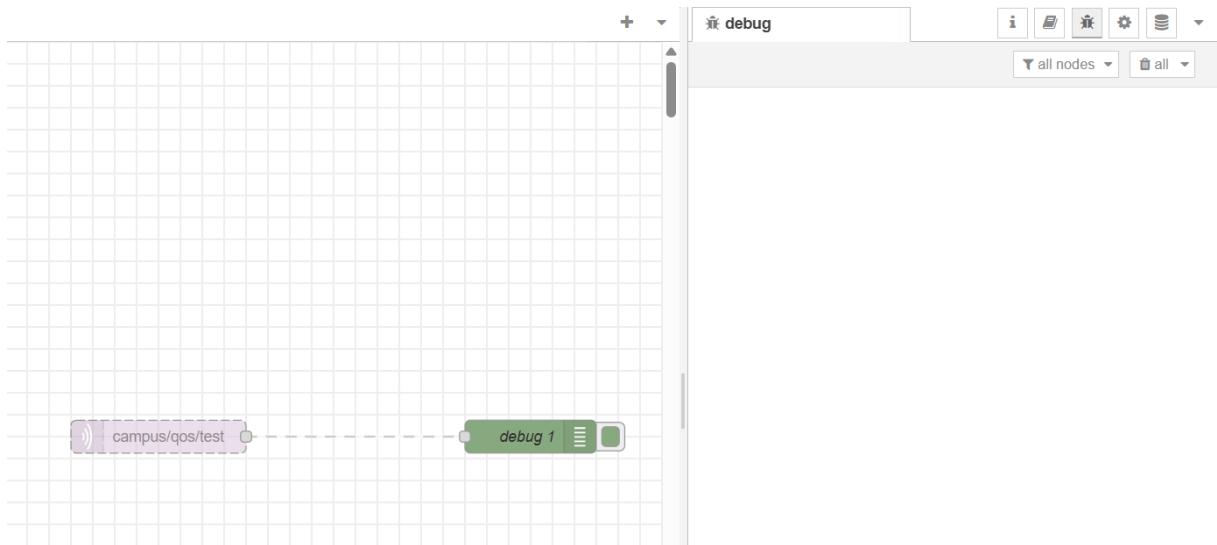


Figure 11: Disconnected the Nodered

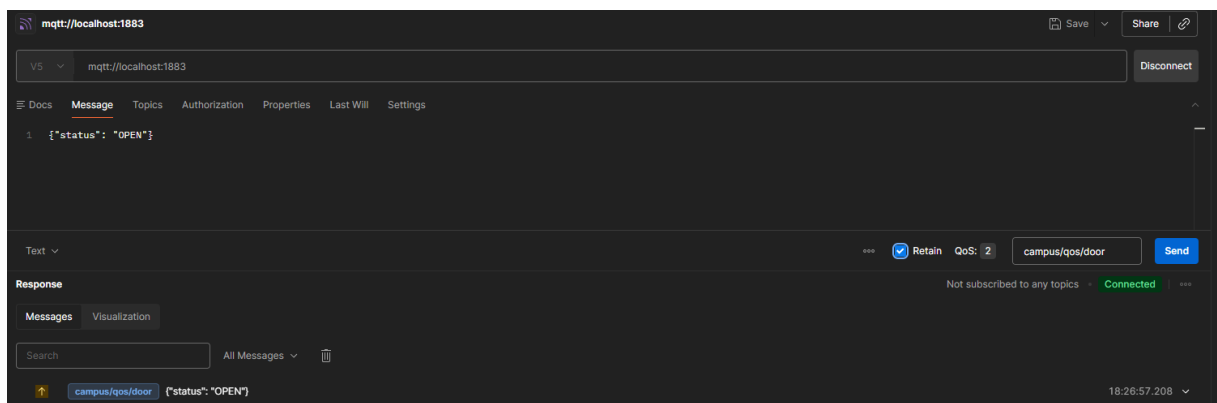


Figure 12: Postman Configuration for

Capturing from Adapter for loopback traffic capture

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

mqtt

No.	Time	Source	Destination	Protocol	Length Info
355	22.255354	127.0.0.1	127.0.0.1	MQTT	46 Ping Response
591	34.424506	::1	::1	MQTT	66 Ping Request
593	34.425362	::1	::1	MQTT	66 Ping Response
675	42.021158	::1	::1	MQTT	104 Publish Message (id=3082) [campus/qos/door]
685	42.021984	::1	::1	MQTT	70 Publish Received (id=3082)
691	42.022290	127.0.0.1	127.0.0.1	MQTT	83 Publish Message (id=1) [campus/qos/door]
711	42.024877	127.0.0.1	127.0.0.1	MQTT	48 Publish Release (id=1)
715	42.036341	::1	::1	MQTT	70 Publish Release (id=3082)
717	42.036718	::1	::1	MQTT	70 Publish Complete (id=3082)

Frame 675: Packet, 104 bytes on wire (832 bits), 104 bytes captured (832 bits) on interface \Device\NPF_{...} id 0

Null/Loopback

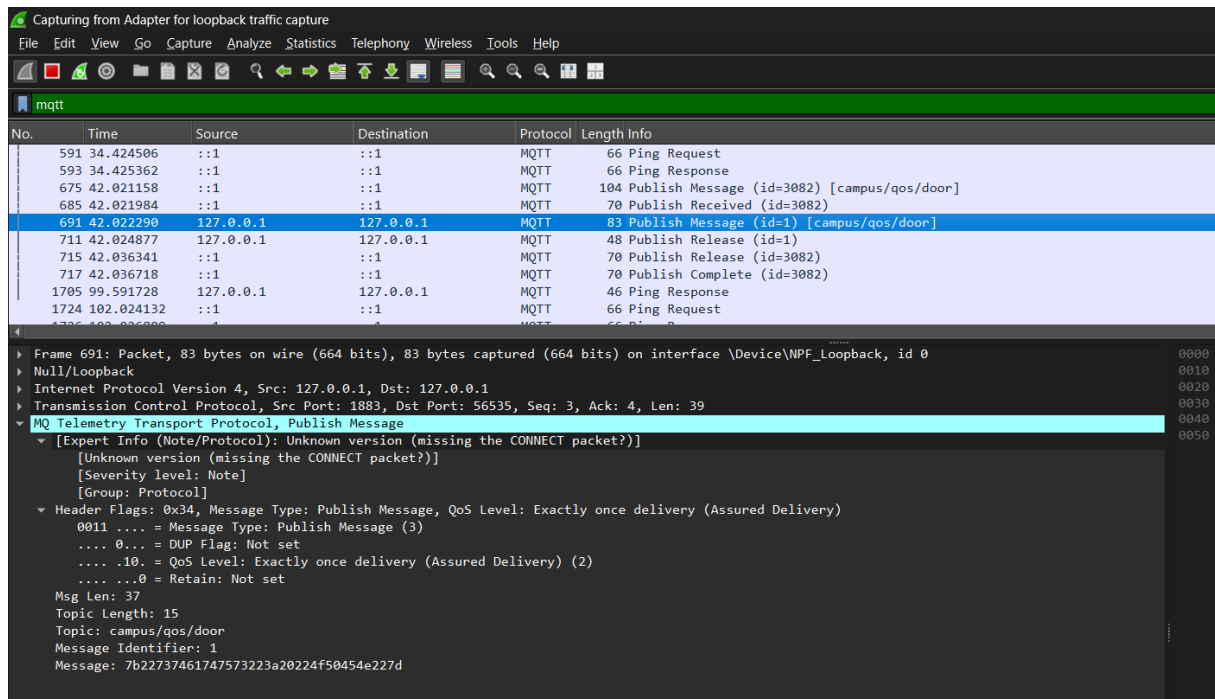
Internet Protocol Version 6, Src: ::1, Dst: ::1

Transmission Control Protocol, Src Port: 56810, Dst Port: 1883, Seq: 3, Ack: 3, Len: 40

MQ Telemetry Transport Protocol, Publish Message

- [Expert Info (Note/Protocol): Unknown version (missing the CONNECT packet?)]
- [Unknown version (missing the CONNECT packet?)]
- [Severity level: Note]
- [Group: Protocol]
- Header Flags: 0x35, Message Type: Publish Message, QoS Level: Exactly once delivery (Assured Delivery), Retain
- 0011 = Message Type: Publish Message (3)
- 0... = DUP Flag: Not set
- 10.. = QoS Level: Exactly once delivery (Assured Delivery) (2)
-1 = Retain: Set
- Msg Len: 38
- Topic Length: 15
- Topic: campus/qos/door
- Message Identifier: 3082
- Message: 007b22737461747573223a20224f50454e227d

Figure 13: Status of Retain bit



No.	Time	Source	Destination	Protocol	Length	Info
591	34.424506	::1	::1	MQTT	66	Ping Request
593	34.425362	::1	::1	MQTT	66	Ping Response
675	42.021158	::1	::1	MQTT	104	Publish Message (id=3082) [campus/qos/door]
685	42.021984	::1	::1	MQTT	70	Publish Received (id=3082)
691	42.022290	127.0.0.1	127.0.0.1	MQTT	83	Publish Message (id=1) [campus/qos/door]
711	42.024877	127.0.0.1	127.0.0.1	MQTT	48	Publish Release (id=1)
715	42.036341	::1	::1	MQTT	70	Publish Release (id=3082)
717	42.036718	::1	::1	MQTT	70	Publish Complete (id=3082)
1705	99.591728	127.0.0.1	127.0.0.1	MQTT	46	Ping Response
1724	102.024132	::1	::1	MQTT	66	Ping Request

Frame 691: Packet, 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface \Device\NPF_{...}_id 0

Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 1883, Dst Port: 56535, Seq: 3, Ack: 4, Len: 39

MQ Telemetry Transport Protocol, Publish Message

- [Expert Info (Note/Protocol): Unknown version (missing the CONNECT packet?)]
- [Unknown version (missing the CONNECT packet?)]
- [Severity level: Note]
- [Group: Protocol]
- Header Flags: 0x34, Message Type: Publish Message, QoS Level: Exactly once delivery (Assured Delivery)
 - 0011 = Message Type: Publish Message (3)
 - 0... = DUP Flag: Not set
 - 10.. = QoS Level: Exactly once delivery (Assured Delivery) (2)
 -0 = Retain: Not set
- Msg Len: 37
- Topic Length: 15
- Topic: campus/qos/door
- Message Identifier: 1
- Message: 7b22737461747573223a202224f50454e227d

Figure 14: Wireshark Traffic

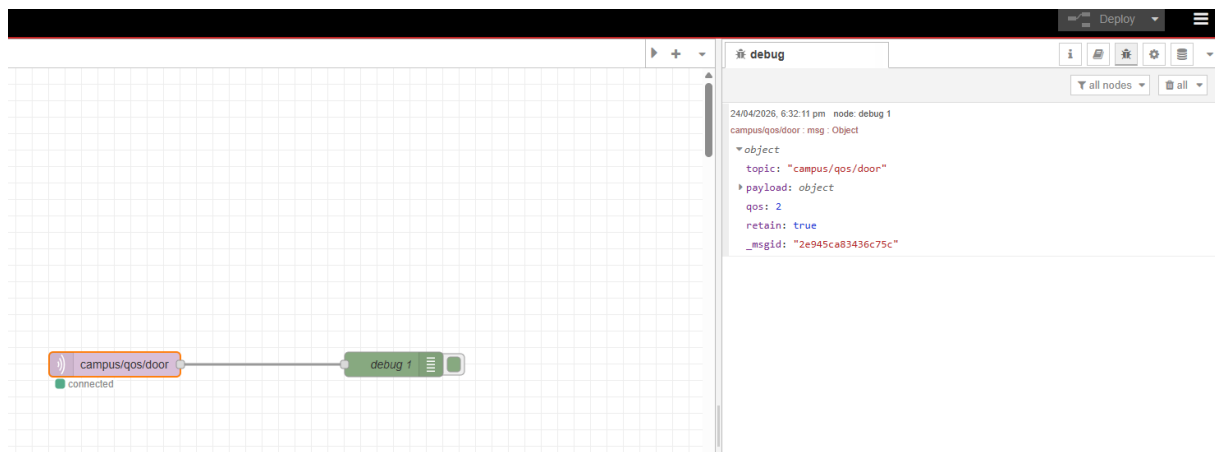


Figure 15: Connect back the Nodered with Debug

2746	168.481304	127.0.0.1	127.0.0.1	MQTT	48 Unsubscribe Ack (id=37497)
2759	168.504201	127.0.0.1	127.0.0.1	MQTT	81 Connect Command
2761	168.506628	127.0.0.1	127.0.0.1	MQTT	48 Connect Ack
2763	168.510074	127.0.0.1	127.0.0.1	MQTT	66 Subscribe Request (id=62982) [campus/qos/door]
2765	168.511051	127.0.0.1	127.0.0.1	MQTT	49 Subscribe Ack (id=62982)
2767	168.511161	127.0.0.1	127.0.0.1	MQTT	83 Publish Message (id=1) [campus/qos/door]
2769	168.512461	127.0.0.1	127.0.0.1	MQTT	48 Publish Received (id=1)
2771	168.513111	127.0.0.1	127.0.0.1	MQTT	48 Publish Release (id=1)
2773	168.514159	127.0.0.1	127.0.0.1	MQTT	48 Publish Complete (id=1)
3560	222.045660	:::1	:::1	MQTT	66 Ping Request
3562	222.046616	:::1	:::1	MQTT	66 Ping Response

```

> Frame 2767: Packet, 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface \Device\NPF_{...} id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 1883, Dst Port: 63569, Seq: 10, Ack: 60, Len: 39
> MQ Telemetry Transport Protocol, Publish Message
  > Header Flags: 0x35, Message Type: Publish Message, QoS Level: Exactly once delivery (Assured Delivery), Retain
    0011 .... = Message Type: Publish Message (3)
    .... 0... = DUP Flag: Not set
    .... 10.. = QoS Level: Exactly once delivery (Assured Delivery) (2)
    .... ...1 = Retain: Set
  Msg Len: 37
  Topic Length: 15
  Topic: campus/qos/door
  Message Identifier: 1
  Message: 7b22737461747573223a202224f50454e227d

```

Figure 16 : Latest Status

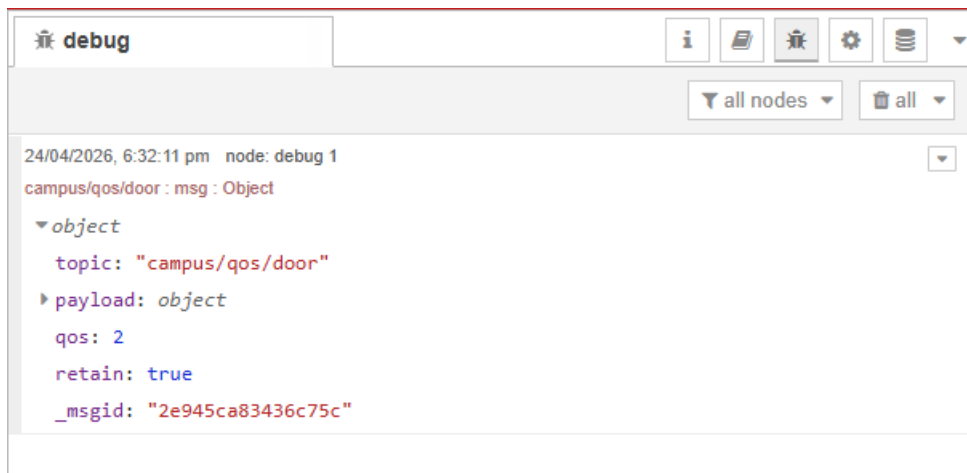


Figure 17: Debug

Discussion

1. As observed that QoS 0 requires 1 packet, QoS 1 requires 2 packets, and QoS 2 requires 4 packets. If you are designing a battery-powered ESP32 temperature sensor that updates every 10 seconds, which QoS level would you choose and why? What are the mathematical consequences on battery life and bandwidth if you incorrectly select QoS 2 for this scenario?

For a battery-powered ESP32 temperature sensor that updates every 10 seconds, **QoS 0** is the best choice because it only requires one packet per message, minimizing power and bandwidth usage, and occasional data loss is acceptable since a new reading arrives quickly. Mathematically, the sensor sends 6 messages per minute ($60 \div 10$), so QoS 0 uses 6 packets per minute, while QoS 2 uses 24 packets per minute (4 packets per message), which is **4x more traffic**; this directly increases Wi-Fi transmission time and energy consumption, meaning the battery could drain significantly faster (e.g., communication lifetime dropping from around 40 hours to about 10 hours), making QoS 2 inefficient and unnecessary for this scenario.

2. Explain the fundamental difference between standard MQTT message delivery and a Retained message. Give a real-world Industrial IoT (IIoT) scenario where a Retained message is mandatory for system safety when a new client connects to the broker.

The fundamental difference is that standard MQTT messages are only delivered to clients that are connected at the time of publishing, while a **Retained message** is stored by the broker and automatically sent to any new subscriber when they connect, ensuring they immediately receive the latest state. In an Industrial IoT scenario such as a factory gas monitoring system, a retained message is critical for safety because if a gas sensor publishes a "DANGER" status with the retain flag, any newly connected monitoring system or alarm controller will instantly know the hazardous condition, instead of waiting for the next update and potentially missing a critical warning.